

BCA203P- Python Programming

Part A

1. Write a python Program To find “Anagram” and “Palindrome”

Code:

```
print("****Anagram****")
s1 =input("Enter string1 ")
s2 =input("Enter string2")

def anagram_check(s1, s2):
    if(sorted(s1)== sorted(s2)):
        print("The strings are anagrams.")
    else:
        print("The strings aren't anagrams.")
anagram_check(s1, s2)

print("****palindrome****")
p1=input("Enter string to check palindrome ")
def palind_check(p1):
    rev_p=p1[::-1]
    if p1==rev_p:
        print("It's a palindrome")
    else:
        print("It's not a palindrome")

palind_check(p1)
```

2. Write a python program to Demonstrate linear search.

Code:

```
list1=[]

#input the number of elements
n=int(input("enter the number of elements: "))

print("Enter the elements")

for i in range(n):
    ele=int(input())
    list1.append(ele)

print("list elemens are : " , list1)

# input the key to be searched
key=int(input("enter the key to be searched"))

def linear_Search(list1,n,key):
    for i in range(0,n):
        if list1[i]==key:
            return i
    return -1

print("_____Linear Search_____")

#perform the linear search
res=linear_Search(list1,n,key

if res== -1:
    print("elements not found")
else:
    print("elements found  at index : " , res)
```

3. Write a Menu Driven Program to add and Delete Stationary items. Use a Dictionary for Adding Items and Brands.

- i. 1.Add an item to the stationaries**
- ii. 2.Remove an item from stationaries**
- iii. 3.Display stationery**
- iv. 4.exit**

Code:

```
dict={}
print("Choose the Option from the Menu")
print("-----")
print("1.Add an item to the stationaries ")
print("2.Remove an item from stationaries ")
print("3.Display stationery ")
print("4.exit")
#key - brand
#value - item
ch=int(input("Enter your Choice: "))
while(ch!=5):
    if ch==1:
        brand=input("Enter the brand [key] to be inserted : ")
        item=input("Enter the item to be inserted : ")
        dict[brand]=item
        print("Added successfully" )
    elif ch==2:
        brand=input("Enter the brand [key] to be deleted: ")
        if brand in dict.keys():
            del dict[brand]
            print(" Deleted successfully" )
        elif brand not in dict.keys():
            print("invalid key")
    elif ch==3:
        print(dict)
    elif ch==4:
        print( "Exiting!!!!!!")
        break
    else:
        print("Invalid choice....!! Choose option between 1 to 4")
ch=int(input("Enter your choice: "))
```

4. Write a Python Program to Find the smallest element greater than K using list comprehension.

```
lst = []

#adding item into list

n = int(input("Enter number of elements : "))

print ("The", n ,"element in ascending order")

for i in range(0, n):

    ele = int(input())

    # adding the element

    lst.append(ele)

print("Given list : ",lst)

#finding the smallest of value after k

k = int(input("enter the K value "))

res = min(i for i in lst if i > k)

print("The smallest element greater than k is : \n" ,res)
```

5. Write a Python program ,

a. To remove empty tuples from the list.

b. Find the largest and smallest element from the given list

Code (a):

```
tuples = [(), ('alex', '15', '8'), (), ('jhon', 'roy'),('tom', 'joy', '45'), ('', ''), ()]
print(tuples)

def Remove(tuples):
    for i in tuples:
        if(len(i) == 0):
            tuples.remove(i)

    return tuples

print("after removing the empty tuples from the list")
print(Remove(tuples))
```

code(b):

```
lst = []
num = int(input('enter the number of elements'))
for n in range(num):
    numbers = int(input('Enter number'))
    lst.append(numbers)
print(lst)
print("Maximum element in the list is :", max(lst))
print("Minimum element in the list is :", min(lst))
```

6. Write a User Input Program to Check if a given number is a Prime Number or not

Code:

```
num = int(input("Enter a number"))
if num == 1:
    print("it is not prime")

if num > 1:
    for i in range(2, num):
        if num % i == 0:
            print("The number ", num, "is not a prime")
            break
    else:
        print("The number ", num, "is a prime")
```

- 7. Write a python program to,**
- a. To find sum of digit of a given number**
 - b. To Find the Sum of n Natural numbers**

CODE:

- a. To find sum of digit of a given number**

```
num = input("Enter Number: ")
```

```
sum = 0
```

```
for i in num:
```

```
    sum = sum + int(i)
```

```
print(sum)
```

- b. Find the Sum of n Natural numbers**

Natural numbers are all positive integers from 1 to infinity.

CODE:

```
n=int(input("Enter a number: "))
```

```
sum1 = 0
```

```
while(n > 0):
```

```
    sum1=sum1+n
```

```
    n=n-1
```

```
print("The sum of first n natural numbers is",sum1)
```

8. Write a Python program to add two matrices.

Code:

```
X = [[1,2,3],
      [4,5,6],
      [7,8,9]]
Y = [[10,11,12],
      [13,14,15],
      [16,17,18]]
result = [[0,0,0],
           [0,0,0],
           [0,0,0]]

# iterate through rows
for i in range(len(X)):
    for j in range(len(X[0])):
        result[i][j] = X[i][j] + Y[i][j]

for r in result:
    print(r)
```

9. Write a multithreaded program to create 2 threads where one thread calculates the factorial and second thread calculates square of a number.

CODE:

```
import threading

#function to calculate factorial
def cal_fact(num):
    fact=1
    for i in range (1, num+1):
        fact=fact*i
    print(f'Factorial of {num} is {fact}')

#function to calculate square.
def cal_square(num):
    square=num*num
    print(f'\n square of the {num} is {square}')

#Main function to create threads
def main():
    num=int(input("enter the number"))
    # creating two threads
    thread1= threading.Thread(target=cal_fact,args=(num,))
    thread2= threading.Thread(target=cal_square,args=(num,))
    # starting the thread
    thread1.start()
    thread2.start()

    #waiting for both thread to finish
    thread1.join()
    thread2.join()

    print("Both threads are executed")
if __name__=="__main__":
    main()
```

10. Write a Python program that opens a file and handles a `FileNotFoundError` exception if the file does not exist.

```
def open_file(filename):  
    try:  
        file = open(filename, 'r')  
        contents = file.read()  
        print("File contents:")  
        print(contents)  
        file.close()  
    except FileNotFoundError:  
        print("Error: File not found.")  
  
file_name = input("Input a file name: ")  
open_file(file_name)
```

PART B:

1. Write a python program to create a GUI application to implement road signs with the appropriate foreground and background colours based on sign type stop, wait and Go signals.

Code :

```
import tkinter as tk
class RoadSignApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Road Signs")

        # Create buttons for each road sign type
        self.create_buttons()

        # Label to display the current sign
        self.sign_label = tk.Label(self.root, text="", font=("Arial", 24))
        self.sign_label.pack(pady=20)

    def create_buttons(self):
        # Stop Sign Button
        stop_button = tk.Button(self.root, text="Stop",
                                command=self.show_stop_sign)
        stop_button.pack(pady=10, padx=10)

        # Wait Sign Button
        wait_button = tk.Button(self.root, text="Wait",
                                command=self.show_wait_sign)
        wait_button.pack(pady=10, padx=10)

        # Go Sign Button
        go_button = tk.Button(self.root, text="Go",
                               command=self.show_go_sign)
        go_button.pack(pady=10, padx=10)

    def show_stop_sign(self):
        self.sign_label.config(text="STOP", fg="white", bg="red")
        self.root.config(bg="red")

    def show_wait_sign(self):
        self.sign_label.config(text="WAIT", fg="black", bg="yellow")
```

```

self.root.config(bg="yellow")

def show_go_sign(self):
    self.sign_label.config(text="GO", fg="white", bg="green")
    self.root.config(bg="green")

if __name__ == "__main__":
    root = tk.Tk()
    app = RoadSignApp(root)
    root.geometry("300x300")
    root.mainloop()

```

output:



2. Write a Python program to create a GUI application using Tkinter that controls the text font size in a label using a slider. Additionally, include a dropdown to change the font colour and a button to randomly change the window's background colour.

CODE:

```
import tkinter as tk
from tkinter import ttk
import random

class FontSizeApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Font Size Controller")

        # Default settings
        self.default_font_size = 16
        self.default_font_color = "black"
        self.default_bg_color = "white"

        # Create a label to display the text
        self.label = tk.Label(self.root, text="Welcome to Yenepoya",
                               font=("Arial", self.default_font_size), fg=self.default_font_color)
        self.label.pack(pady=20)

        # Create a slider to control font size
        self.font_size_slider = tk.Scale(self.root, from_=10, to=100,
                                          orient="horizontal", label="Font Size", command=self.update_font_size)
        self.font_size_slider.set(self.default_font_size)
        self.font_size_slider.pack(pady=10)

        # Dropdown to change text color
        self.color_label = tk.Label(self.root, text="Choose Font Color:")
        self.color_label.pack(pady=5)
        self.color_var = tk.StringVar(value="black")
        self.color_dropdown = ttk.Combobox(self.root,
                                           textvariable=self.color_var, values=["black", "red", "blue", "green",
                                           "purple"], state="readonly")
        self.color_dropdown.pack(pady=5)
        self.color_dropdown.bind("<<ComboboxSelected>>",
                                self.update_color)

        # Button to change background color randomly
```

```

        self.bg_button = tk.Button(self.root, text="Change Background Color",
command=self.change_bg_color)
        self.bg_button.pack(pady=10)

    def update_font_size(self, size):
        # Update the font size of the label
        self.label.config(font=("Arial", int(size)))

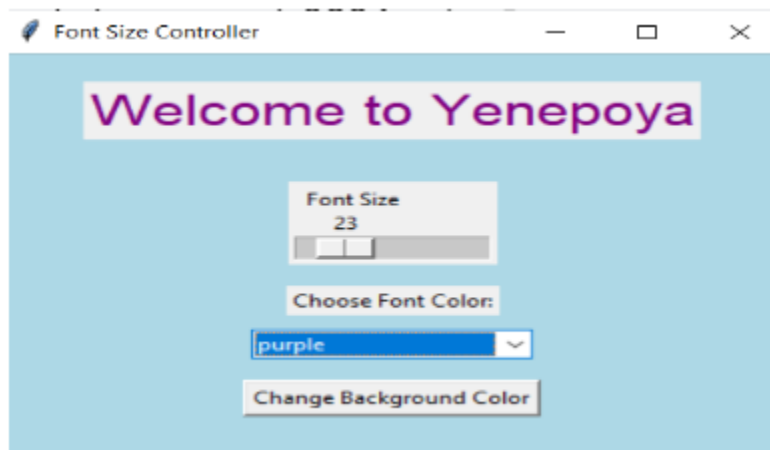
    def update_color(self, event):
        # Update the font color of the label
        self.label.config(fg=self.color_var.get())

    def change_bg_color(self):
        # Change the background color to a random one
        random_color = random.choice(["lightblue", "lightgreen",
"lightyellow", "lightpink", "lightgray"])
        self.root.config(bg=random_color)

if __name__ == "__main__":
    root = tk.Tk()
    app = FontSizeApp(root)
    root.geometry("400x300")
    root.mainloop()

```

OUTPUT:



3. Write a python script that performs the following operations on the SQLite database:

- a. Creates a students table.
- b. Inserts two records into the table.
- c. Retrieves and displays the records.
- d. Updates a student's grade.
- e. Deletes a student record.
- f. Closes the database connection.

CODE:

```
import sqlite3

# Step 1: Connect to SQLite database (or create it if it doesn't exist)

conn = sqlite3.connect('students.db')

cursor = conn.cursor()

# Step 2: Create the students table

cursor.execute("""

CREATE TABLE IF NOT EXISTS students (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    name TEXT NOT NULL,

    grade INTEGER NOT NULL)""")

print("table is created successfully")

# Step 3: Insert two records into the students table

cursor.execute("INSERT INTO students (name, grade) VALUES ('Alice', 90)")

cursor.execute("INSERT INTO students (name, grade) VALUES ('Bob', 85)")

print("2 records are inserted into table")

# Commit the changes

conn.commit()
```

Step 4: Retrieve and display all records

```
cursor.execute("SELECT * FROM students")
```

```
print("Students before update:")
```

```
rows = cursor.fetchall()
```

```
for row in rows:
```

```
    print(row)
```

Step 5: Update the grade of a student (e.g., update Bob's grade to 95)

```
cursor.execute("UPDATE students SET grade = 95 WHERE name = 'Bob'")
```

Commit the changes

```
conn.commit()
```

Step 6: Retrieve and display the updated records

```
cursor.execute("SELECT * FROM students")
```

```
print("\nStudents after update:")
```

```
rows = cursor.fetchall()
```

```
for row in rows:
```

```
    print(row)
```

Step 7: Delete a student record (e.g., delete Alice)

```
cursor.execute("DELETE FROM students WHERE name = 'Alice'")
```

```
# Commit the changes
```

```
conn.commit()
```

```
# Step 8: Retrieve and display the records after deletion
```

```
cursor.execute("SELECT * FROM students")
```

```
print("\nStudents after deletion:")
```

```
rows = cursor.fetchall()
```

```
for row in rows:
```

```
    print(row)
```

```
# Step 9: Close the database connection
```

```
conn.close()
```

4. Write a Python program to perform the following tasks related to employee data using Pandas:

1. Create a DataFrame containing the following details of employees:
 - Name, Age, Salary, Department(IT, HR, Finance).
2. Save the DataFrame to a CSV file named employees.csv.
3. Read the DataFrame from the employees.csv file.
4. Perform the following operations on the DataFrame:
 - Indexing: Select and display the Age column.
 - Selection: Select and display the Name and Department columns for all employees.
 - Filtering:
 - Display all employees who are older than 30.
 - Display all employees who work in the "IT" department.

Code:

```
import pandas as pd
# Step 1: Create a DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [23, 35, 45, 27, 32],
    'Salary': [60000, 52000, 72000, 48000, 91000],
    'Department': ['HR', 'IT', 'Finance', 'IT', 'HR']
}
df = pd.DataFrame(data)

# Step 2: Write the DataFrame to a CSV file
df.to_csv('employees.csv', index=False)
print("DataFrame saved to 'employees.csv'.")

# Step 3: Read the DataFrame from the CSV file
df_read = pd.read_csv('employees.csv')
print("\nDataFrame read from 'employees.csv':")
print(df_read)

# Step 4: Data Indexing, Selection, and Filtering
# Indexing: Select a specific column (Age)
print("\nIndexing: Select 'Age' column:")
print(df['Age'])

# Selection: Select multiple columns (Name and Department)
print("\nSelection: Select 'Name' and 'Department' columns:")
print(df[['Name', 'Department']])
```

```
# Row selection by index (select row at index 2)
print("\nSelect row at index 2:")
print(df.iloc[2])

# Filtering: Select rows where Age is greater than 30
print("\nFiltering: Rows where Age > 30:")
print(df[df['Age'] > 30])

# Filtering: Select rows where Department is 'IT'
print("\nFiltering: Rows where Department is 'IT':")
print(df[df['Department'] == 'IT'])
```

5. Write a python program to create two threads to count how many lines in two text files (one thread will count lines from the first file and other thread from the second file).

```
import threading

# Function to count lines in a given file
def count_lines(file_name):
    try:
        with open(file_name, 'r') as file:
            lines = file.readlines()
            print(f"{file_name} contains {len(lines)} lines. \n")
    except FileNotFoundError:
        print(f"File {file_name} not found!")

# File names
file1 = 'file1.txt'
file2 = 'file2.txt'

# Step 1: Create two threads, one for each file
thread1 = threading.Thread(target=count_lines, args=(file1,))
thread2 = threading.Thread(target=count_lines, args=(file2,))

# Step 2: Start both threads
thread1.start()
thread2.start()

# Step 3: Wait for both threads to complete
thread1.join()
thread2.join()

print("Line counting completed for both files.")
```